

imageprocessing

May 12, 2026

```
[1]: # Cell 1 - Imports
from io import BytesIO
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

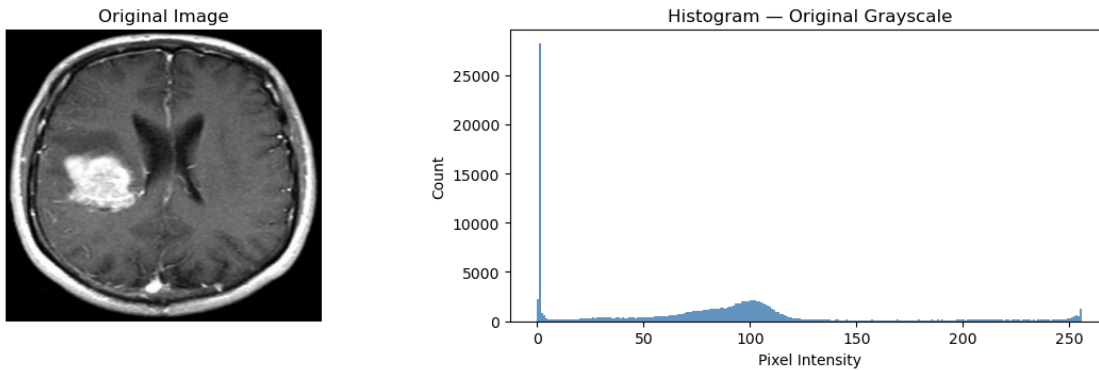
[2]: # Cell 2 - Load Image
IMAGE_PATH = "./test_images/yes/Y11.jpg"

img_pil = Image.open(IMAGE_PATH).convert("RGB")
img_arr = np.array(img_pil, dtype=np.float32) / 255.0 # float32 [0,1]
img_u8 = (img_arr * 255).astype(np.uint8) # uint8 [0,255] RGB
img_bgr = cv2.cvtColor(img_u8, cv2.COLOR_RGB2BGR) # BGR for OpenCV
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY) # grayscale

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].imshow(img_u8)
axes[0].set_title("Original Image", fontsize=12)
axes[0].axis("off")
axes[1].hist(gray.ravel(), bins=256, range=(0,256), color="steelblue", alpha=0.
↪85)
axes[1].set_title("Histogram - Original Grayscale")
axes[1].set_xlabel("Pixel Intensity")
axes[1].set_ylabel("Count")
plt.suptitle("Stage 0 · Original", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

print(f"Shape : {img_u8.shape} | dtype : {img_u8.dtype} | range : [{img_u8.
↪min()}, {img_u8.max()}]")
```

Stage 0 · Original



Shape : (369, 400, 3) | dtype : uint8 | range : [0, 255]

```
[9]: # Cell 3 - Stage 1 : CLAHE Enhancement
clahe      = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
gray_clahe = clahe.apply(gray)
clahe_rgb  = cv2.cvtColor(gray_clahe, cv2.COLOR_GRAY2RGB)

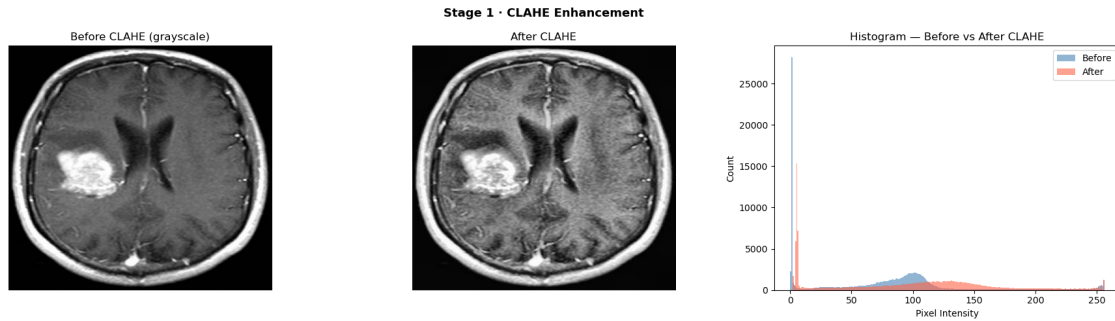
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].imshow(gray, cmap="gray")
axes[0].set_title("Before CLAHE (grayscale)")
axes[0].axis("off")

axes[1].imshow(clahe_rgb)
axes[1].set_title("After CLAHE")
axes[1].axis("off")

axes[2].hist(gray.ravel(), bins=256, range=(0,256), color="steelblue",
             alpha=0.6, label="Before")
axes[2].hist(gray_clahe.ravel(), bins=256, range=(0,256), color="tomato",
             alpha=0.6, label="After")
axes[2].set_title("Histogram - Before vs After CLAHE")
axes[2].set_xlabel("Pixel Intensity")
axes[2].set_ylabel("Count")
axes[2].legend()

plt.suptitle("Stage 1 · CLAHE Enhancement", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()
```



```
[4]: # Cell 4 - Stage 2 : Heatmap (JET colormap)
heatmap_bgr = cv2.applyColorMap(gray, cv2.COLORMAP_JET)
heatmap_rgb = cv2.cvtColor(heatmap_bgr, cv2.COLOR_BGR2RGB)

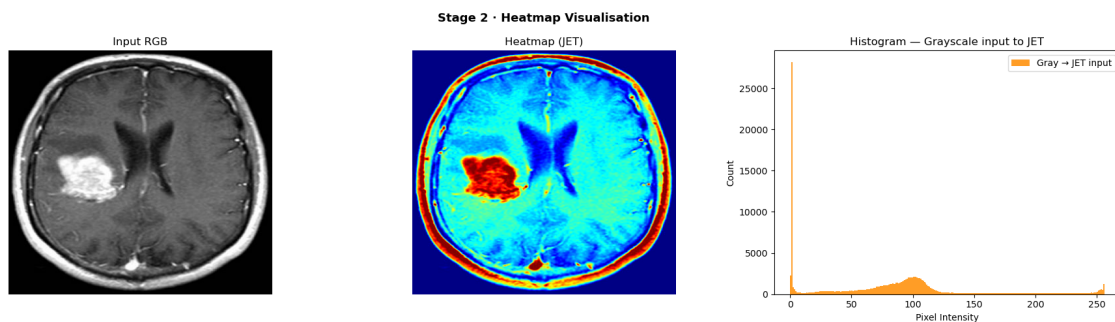
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].imshow(img_u8)
axes[0].set_title("Input RGB")
axes[0].axis("off")

axes[1].imshow(heatmap_rgb)
axes[1].set_title("Heatmap (JET)")
axes[1].axis("off")

axes[2].hist(gray.ravel(), bins=256, range=(0,256), color="darkorange", alpha=0.
    ↪85, label="Gray → JET input")
axes[2].set_title("Histogram - Grayscale input to JET")
axes[2].set_xlabel("Pixel Intensity")
axes[2].set_ylabel("Count")
axes[2].legend()

plt.suptitle("Stage 2 · Heatmap Visualisation", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()
```



```
[5]: # Cell 5 - Stage 3 : Canny Edge Detection
edges      = cv2.Canny(gray, 100, 200)
edges_rgb   = cv2.cvtColor(edges, cv2.COLOR_GRAY2RGB)

unique, counts = np.unique(edges, return_counts=True)
edge_pixel_count = int(counts[unique == 255][0]) if 255 in unique else 0

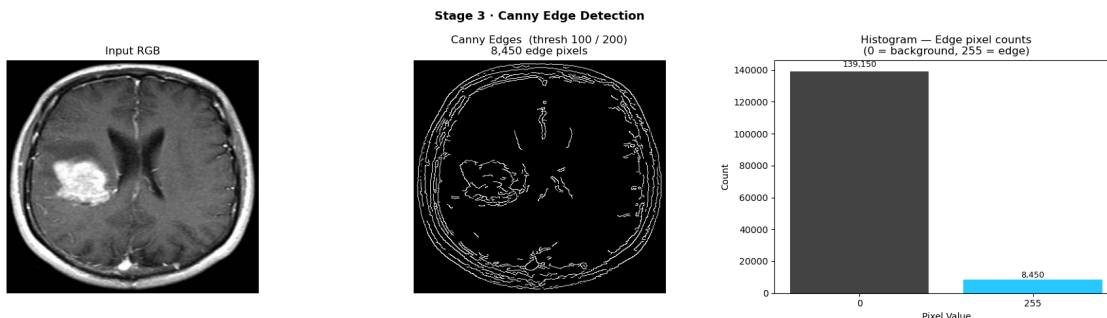
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].imshow(img_u8)
axes[0].set_title("Input RGB")
axes[0].axis("off")

axes[1].imshow(edges_rgb)
axes[1].set_title(f"Canny Edges (thresh 100 / 200)\n{edge_pixel_count:,} edge_
    ↪pixels")
axes[1].axis("off")

bar_labels = [str(v) for v in unique]
bar_colors = ["#222222" if v == 0 else "deepskyblue" for v in unique]
bars = axes[2].bar(bar_labels, counts, color=bar_colors, alpha=0.85)
for bar, cnt in zip(bars, counts):
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height()*1.01,
        f"{cnt:,}", ha="center", va="bottom", fontsize=9)
axes[2].set_title("Histogram - Edge pixel counts\n(0 = background, 255 = edge)")
axes[2].set_xlabel("Pixel Value")
axes[2].set_ylabel("Count")

plt.suptitle("Stage 3 · Canny Edge Detection", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()
```



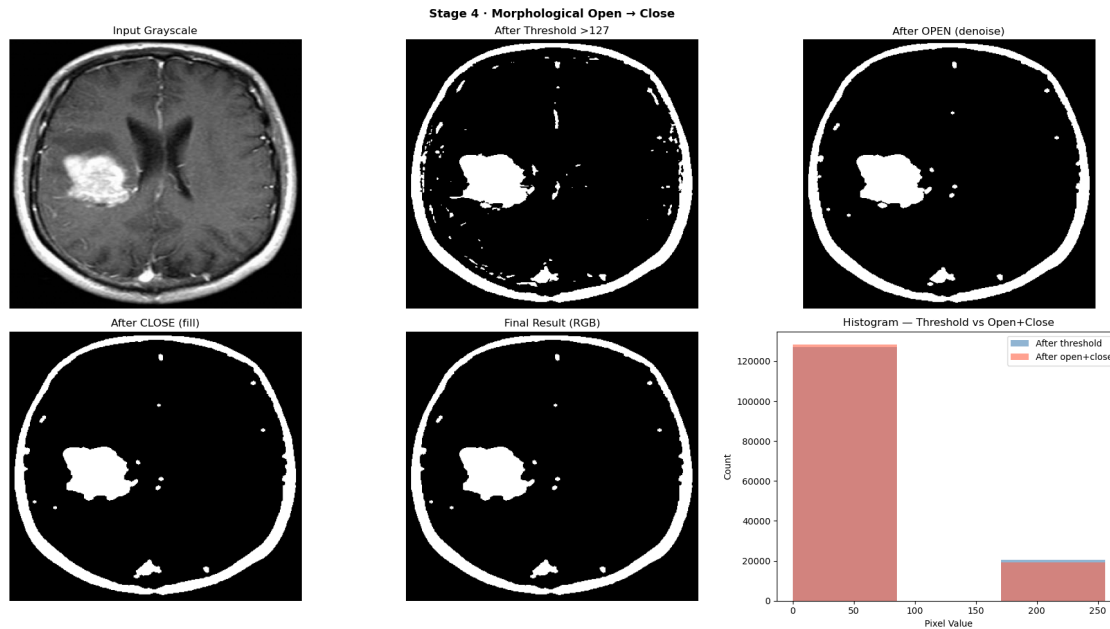
```
[6]: # Cell 6 - Stage 4 : Morphological Open → Close
_, thresh_morph = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
kernel          = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
after_open      = cv2.morphologyEx(thresh_morph, cv2.MORPH_OPEN, kernel)
after_close     = cv2.morphologyEx(after_open,   cv2.MORPH_CLOSE, kernel)
morph_rgb       = cv2.cvtColor(after_close, cv2.COLOR_GRAY2RGB)

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

axes[0,0].imshow(gray,          cmap="gray") ; axes[0,0].set_title("Input_
↳Grayscale")          ; axes[0,0].axis("off")
axes[0,1].imshow(thresh_morph,  cmap="gray") ; axes[0,1].set_title("After_
↳Threshold >127") ; axes[0,1].axis("off")
axes[0,2].imshow(after_open,    cmap="gray") ; axes[0,2].set_title("After OPEN_
↳(denoise)") ; axes[0,2].axis("off")
axes[1,0].imshow(after_close,   cmap="gray") ; axes[1,0].set_title("After CLOSE_
↳(fill)") ; axes[1,0].axis("off")
axes[1,1].imshow(morph_rgb)          ; axes[1,1].set_title("Final Result_
↳(RGB)") ; axes[1,1].axis("off")

axes[1,2].hist(thresh_morph.ravel(), bins=3, range=(0,256), color="steelblue",
↳alpha=0.6, label="After threshold")
axes[1,2].hist(after_close.ravel(), bins=3, range=(0,256), color="tomato",
↳alpha=0.6, label="After open+close")
axes[1,2].set_title("Histogram - Threshold vs Open+Close")
axes[1,2].set_xlabel("Pixel Value")
axes[1,2].set_ylabel("Count")
axes[1,2].legend()

plt.suptitle("Stage 4 · Morphological Open → Close", fontsize=13,
↳fontweight="bold")
plt.tight_layout()
plt.show()
```



```
[7]: # Cell 7 - Stage 5 : Contour Detection
_, thresh_cnt = cv2.threshold(gray, 210, 255, cv2.THRESH_BINARY)
contours, _ = cv2.findContours(thresh_cnt, cv2.RETR_TREE, cv2.
    ↳CHAIN_APPROX_SIMPLE)
contours_sorted = sorted(contours, key=cv2.contourArea, reverse=True)
filtered = [c for c in contours_sorted if cv2.contourArea(c) > 100]

contour_canvas = img_u8.copy()
cv2.drawContours(contour_canvas, filtered, -1, (0, 255, 0), 2)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].imshow(img_u8)
axes[0].set_title("Input RGB")
axes[0].axis("off")

axes[1].imshow(contour_canvas)
axes[1].set_title(f"Contours Detected\n(n = {len(filtered)}, min_area = 100)")
axes[1].axis("off")

axes[2].hist(gray.ravel(), bins=256, range=(0,256), color="slategray", alpha=0.
    ↳85, label="Grayscale")
axes[2].axvline(210, color="red", linewidth=1.8, linestyle="--",
    ↳label="Threshold = 210")
axes[2].set_title("Histogram - Gray + contour threshold")
axes[2].set_xlabel("Pixel Intensity")
```

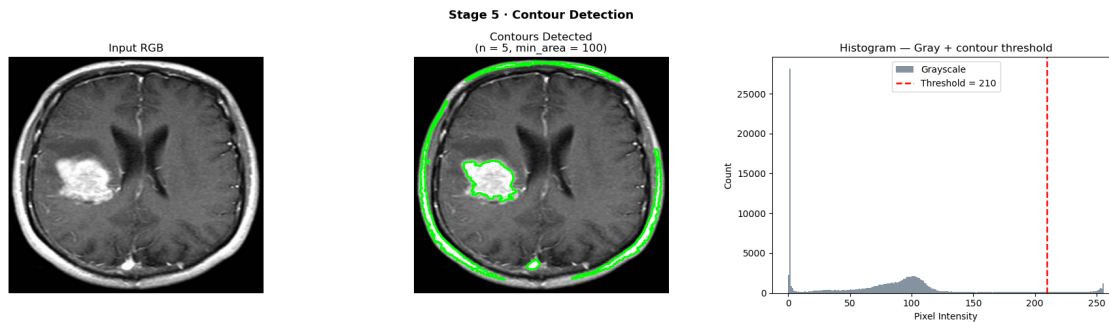
```

axes[2].set_ylabel("Count")
axes[2].legend()

plt.suptitle("Stage 5 · Contour Detection", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

print(f"Total contours found : {len(filtered)}")

```



Total contours found : 5

```

[8]: # Cell 8 - Summary Grid : All Stages Side-by-Side
all_stages = [
    ("Original",          img_u8),
    ("CLAHE Enhanced",    clahe_rgb),
    ("Heatmap (JET)",      heatmap_rgb),
    ("Canny Edges",        edges_rgb),
    ("Morph Open+Close",   morph_rgb),
    ("Contours Detected",   contour_canvas),
]

cols = 3
rows = int(np.ceil(len(all_stages) / cols))

fig, axes = plt.subplots(rows, cols, figsize=(15, 5 * rows))

for ax, (title, img) in zip(axes.flat, all_stages):
    ax.imshow(img)
    ax.set_title(title, fontsize=11)
    ax.axis("off")

for ax in list(axes.flat)[len(all_stages):]:
    ax.set_visible(False)

plt.suptitle("Full Pipeline - All Stages", fontsize=14, fontweight="bold")

```

```
plt.tight_layout()
plt.show()
```

